Secure Sealed-Bid Auction Using 3-Party Arithmetic MPC
Assignment Specification

Project Overview

In this project, you will implement a simplified **privacy-preserving sealed-bid auction** using 3-party arithmetic MPC. The system ensures that:

- Each bidder commits to a bid using an elliptic-curve Pedersen commitment.
- Each bidder generates 3 additive secret shares of their bid modulo a large prime p and sends one share to each server.
- Each bidder provides a simple zero-knowledge (ZK) proof that the bid is within a valid range.
- The three servers compute the maximum bid using a simplified MPC protocol, exchanging only share differences and revealing only the result.

Only the **winner ID** and **winning bid** are revealed. All other bids remain private.

Entities

Bidders

- Generate a commitment to their bid.
- Prove the bid is within a valid range using a ZK proof.
- Secret-share their bid among 3 servers.

Servers (S1, S2, S3)

- Hold additive shares securely.
- Cooperate to compute the maximum bid using MPC operations.
- Return the identity of the winner and the winning amount.

Bidder Procedure

Each bidder follows these steps:

1. **Commit to the bid:** Generate a Pedersen commitment $C = bG + rH$ to hide the bid value.
2. **Zero-Knowledge Proof (ZK Proof):** Generate a non-interactive ZK proof that the bid is within a valid range. For example, valid bids are between **1000** and **50000**. This prevents invalid or malicious bids from entering the auction.
3. **Secret Sharing:** Once the ZK proof is verified, split the bid b into 3 additive shares modulo p: $b = s_1 + s_2 + s_3 \mod p$, sending one share to each of the three servers.

Invalid bids: If a bidder chooses a number outside the valid range (e.g., 999 or 50001), the ZK proof verification fails. The servers reject the bid, and it is not included in the auction.

Cryptographic Techniques

Additive Secret Sharing (mod p)

For a number x, the bidder generates random shares:

$$x = x_1 + x_2 + x_3 \pmod{p}$$

and sends one share to each server: $x_1 \to S_1, x_2 \to S_2, x_3 \to S_3$.

Commitment (EC Pedersen Commitment)

Each bidder commits to their bid b:

$$C = bG + rH$$

Simplified ZK Range Proof

A simple non-interactive ZK proof (e.g. using Fiat–Shamir) that:

$$0 \leq b \leq B_{\max}$$

is sufficient.

Simplified MPC Comparison

To compare secret-shared values, the servers:

1. Locally compute differences between their shares: $\delta_i = x_i - y_i \mod p$
2. Exchange these share differences
3. Reconstruct the **difference of bids** (only the difference)
4. Determine the sign to find the larger bid

This simplification avoids bit-decomposition while illustrating MPC principles.

Project Requirements

Bidder-side Code (Client)

- `commit_bid(bid, randomness)` → `commitment`: Creates a Pedersen commitment.
- `prove_range(bid, randomness, B_max)` → `proof`: Produces a ZK range proof.
- `share_bid(bid, p)` → `(s1, s2, s3)`: Generates additive shares modulo p.

Server-side Code

- `receive_shares(bidder_id, share_value)`: Stores shares securely.
- `compute_difference(a_share, b_share, p)` → `diff_share`: Computes local difference of shares.
- `reconstruct_difference(diff_shares, p)` → `integer`: Reconstructs total difference.
- `compare(a_shares, b_shares, p)` → `-1 or 1`: Determines which bid is larger.
- `find_max_bid(bidders, shares, p)` → `(winner_id, winning_bid)`: Finds maximum bid among all bidders.

Expected Project File Structure

```
project/
  commitments.py
      generate_commitment_params()
      commit_bid()
      verify_commitment()
```

```
zk_proofs.py
    generate_range_proof()
    verify_range_proof()

secret_sharing.py
    share_value()
    reconstruct()

mpc.py
    mpc_add()
    mpc_sub()
    mpc_mult()
    mpc_compare()
    mpc_argmax()

auction.py
    register_bidder()
    run_auction()
```

Example Runs

Acceptable bid range is from 100 to 1000

- **Example 1:** Bidders = 150, 920, 600
  Output: Winner = 2, Winning Bid = 920
- **Example 2:** Bidders = 350, 350, 100, 300
  Output: Winner = 1, Winning Bid = 350
- **Example 3:** Bidders = 10, 999, 300, 700, 2000
  Output: Winner = 2, Winning Bid = 999

Deliverables

- Source code for the project
- Small document explaining how the following integrate together in this project:
  - Secret sharing
  - MPC comparison
  - Commitments and ZK proofs

Please form teams of 3 (cross-labs/TAs is allowed) and be ready for evaluations, this is a group project :)